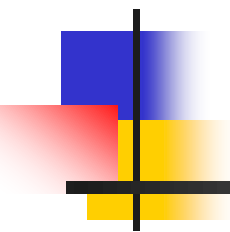
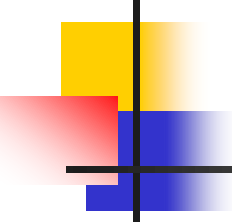


DATA STRUCTURES AND ALGORITHMS



Lecture Notes 5

Pradit Pitaksathienkul



ROAD MAP

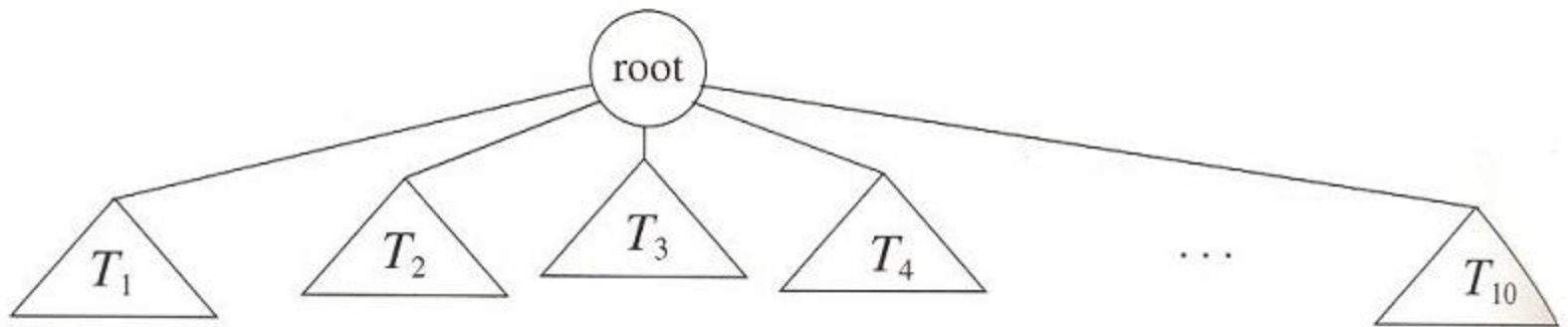
- **TREES**

- **Definitions and Terminology**
- **Implementation of Trees**
- Tree Traversals
- Binary Trees
- The Search Tree ADT-Binary Search Trees

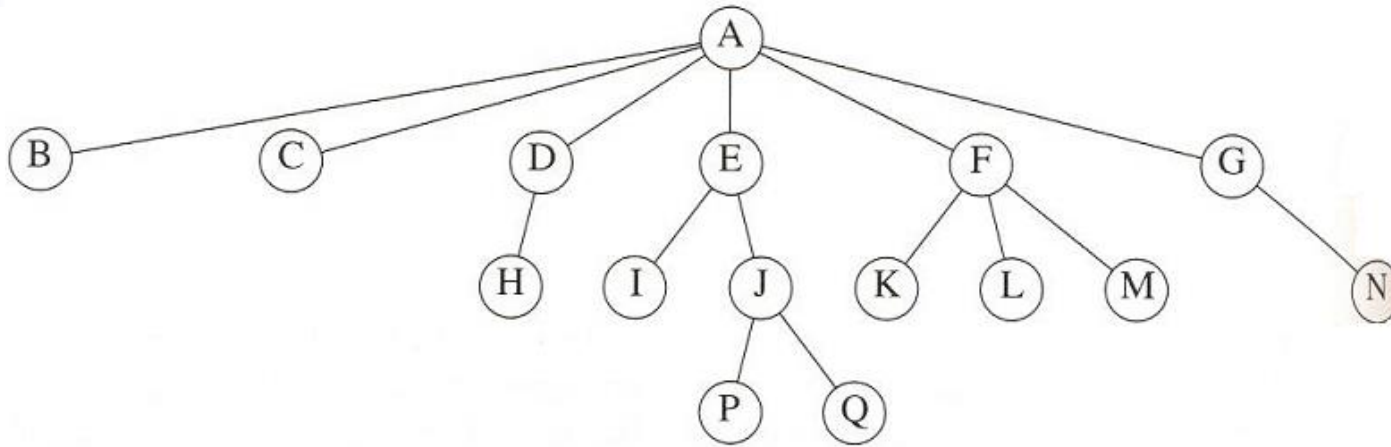
TREES: Definition

Recursive Definition :

- **Tree** is a collection of **nodes**
 - A tree can be empty
 - A tree contains zero or more **subtrees** $T_1, T_2, \dots, T_k$ connected to a **root node** by **edges**

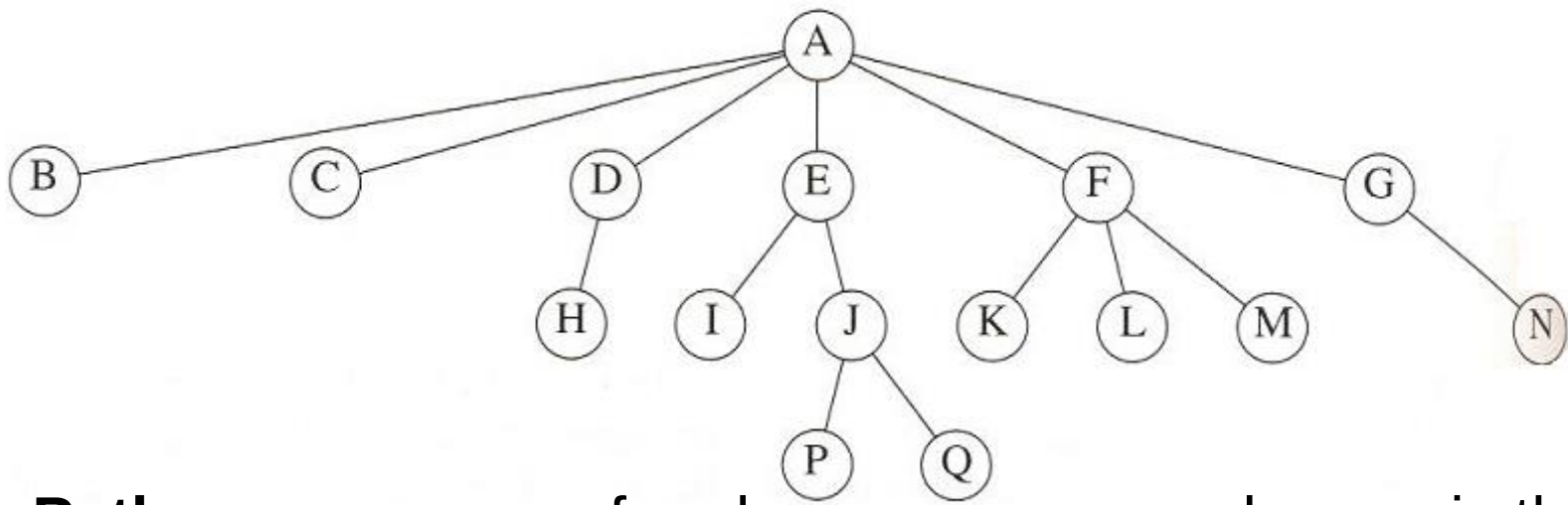


TREES: Terminology



Family Tree Terminology

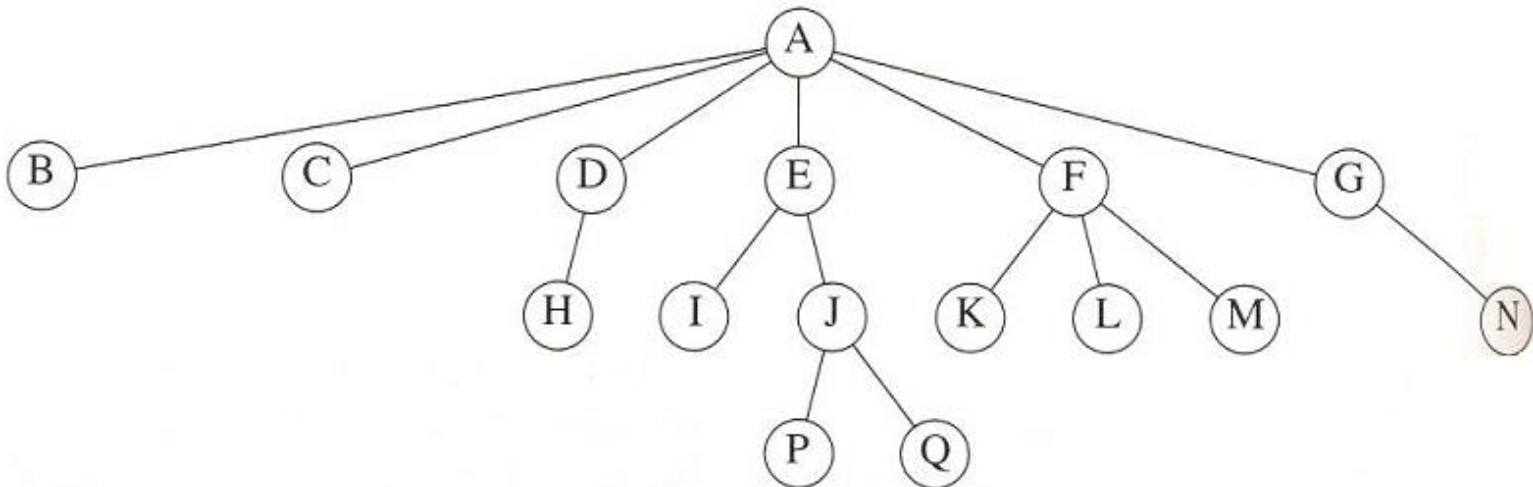
- child → F is child of A
- parent → A is the parent of F
 - each node is connected to a parent **except the root**
- sibling → nodes with same parents (K, L, M)
- leaf → nodes with no children (P, Q)
- Ancestor / Descendant



- **Path** : a sequence of nodes $n_1, n_2, \dots, n_k$, where n_i is the parent of n_{i+1} $1 \leq i < k$, path from node A to node Q is ?
- **Lenght** : number of edges on the path $(k-1)$
- **Depth** : depth of n_i is the lenght of unique path **from the root to n_i**
 - depth of root is 0 , depth of node F is 1, depth of node M is 2
 - depth of a tree = depth of the deepest leaf
- **Height** : height of n_i is the lenght of the longest path **from n_i to a leaf**
 - height of a leaf is 0 , height of node E is 2, height of node A is 3
 - height of a tree = height of the root = depth of the tree

Implementation of Trees

1. Each node contains a pointer to each of its children
 - number of children would be large
2. Each node contains an array of pointers to its children
 - number of children may vary greatly (waste of space)

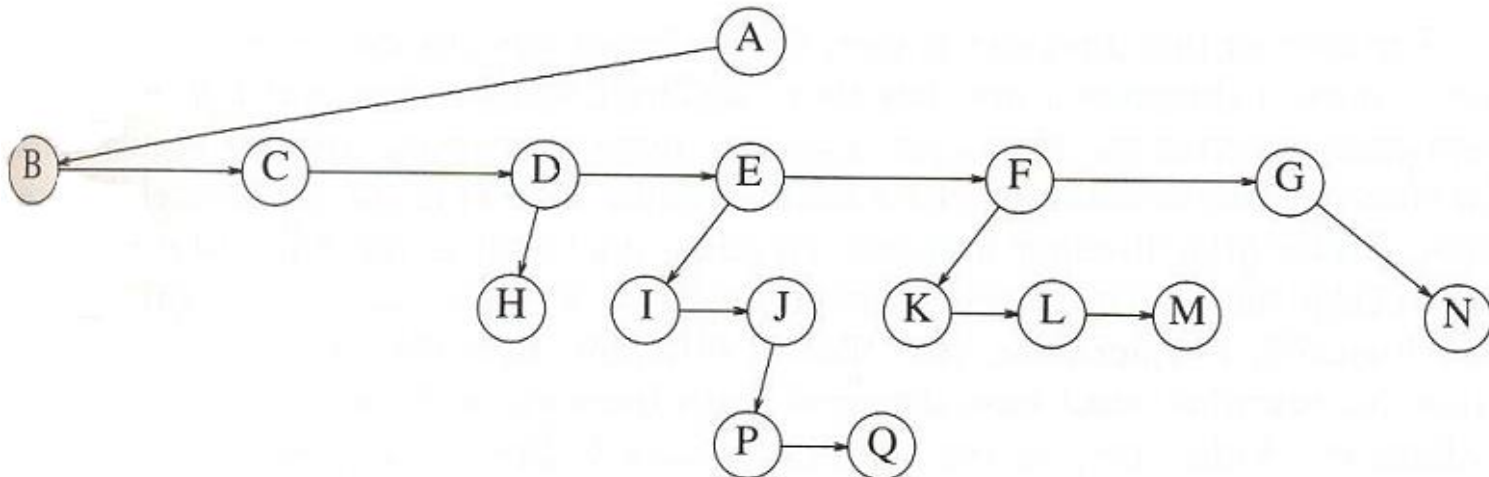


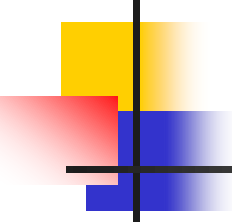
Implementation of Trees

3. Each node contains a linked list of the children

class Node:

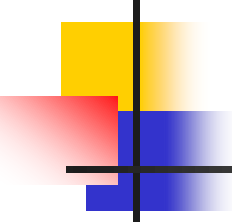
```
def __init__(self, value):  
    self.value = value  
    self.firstChild = None  
    self.nextSibling = None
```





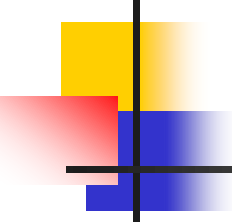
ROAD MAP

- TREES
 - Implementation of Trees
 - **Tree Traversals**
 - Binary Trees
 - The Search Tree ADT-Binary Search Trees



Tree Traversals

- Pre Order: root left right
- In Order: left root right
- Post Order: left right root

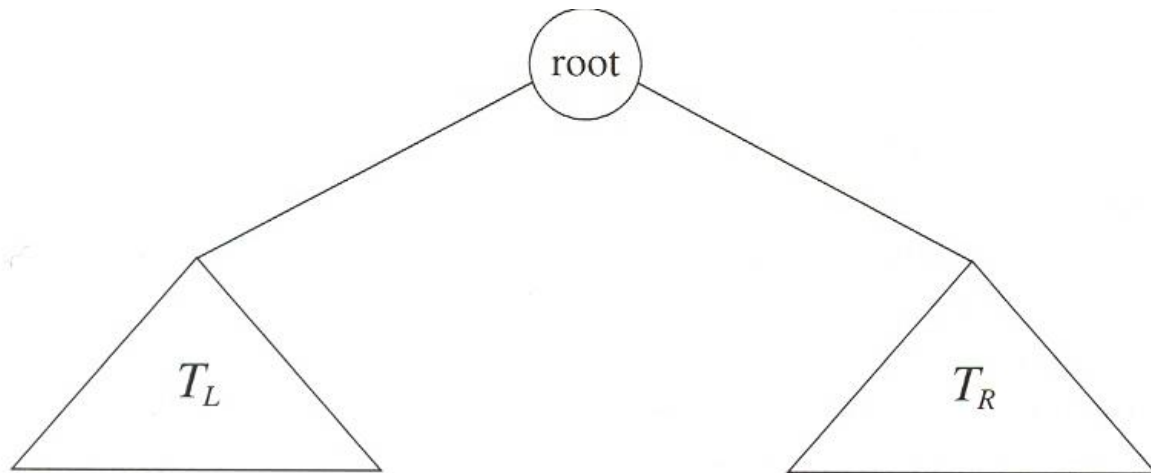


ROAD MAP

- TREES
 - Implementation of Trees
 - Tree Traversals
 - **Binary Trees**
 - The Search Tree ADT-Binary Search Trees

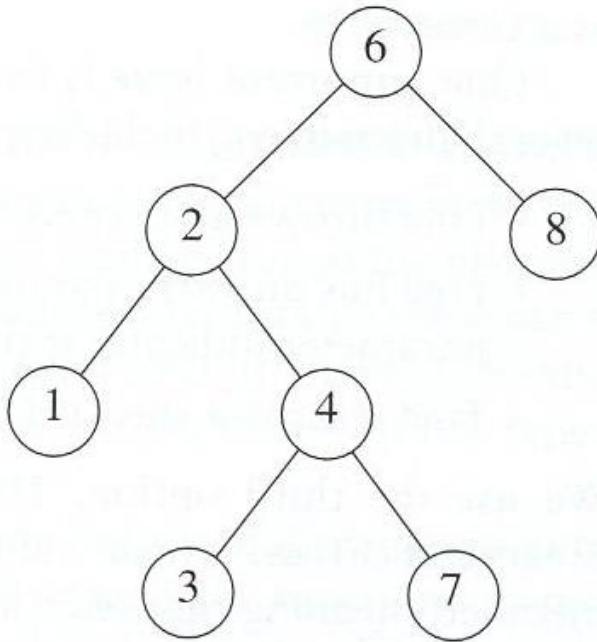
Binary Trees

- **Definition** : A tree in which nodes have at most two children

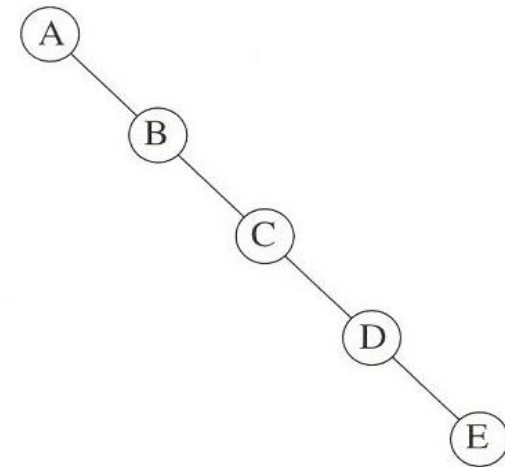


Generic Binary Tree

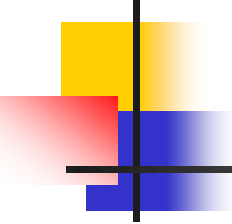
Binary Trees



Binary Tree



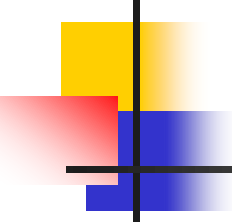
Worst case binary tree



Implementation of Binary Trees

- keep a pointer to left child and right child

```
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
```

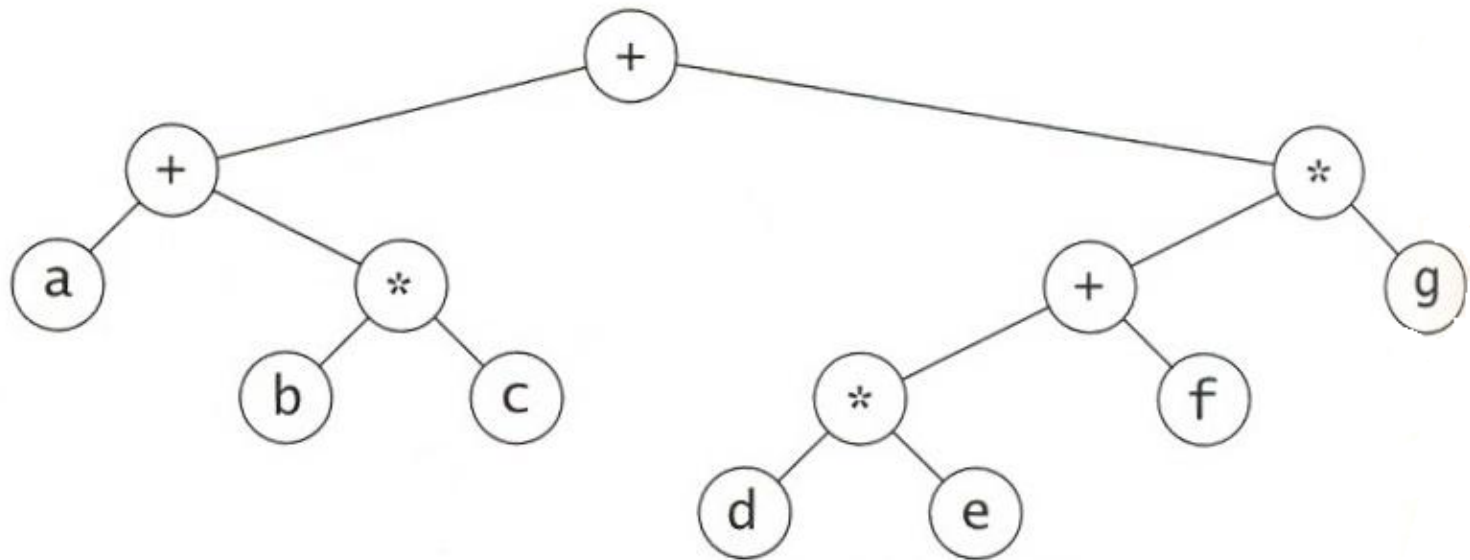


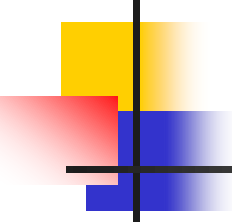
Example: Expression Trees

- Leaves contain operands
- Internal nodes contain operators
- Inorder traversal $\Rightarrow$ infix expression
- Preorder traversal $\Rightarrow$ prefix expression
- Postorder traversal $\Rightarrow$ postfix expression

Expression Trees

- Entire tree represents $(a+(b*c))+(((d*e)+f)*g)$
 - Left subtree represents $a+(b*c)$
 - Right subtree represents $((d*e)+f)*g$





Constructing an Expression Tree

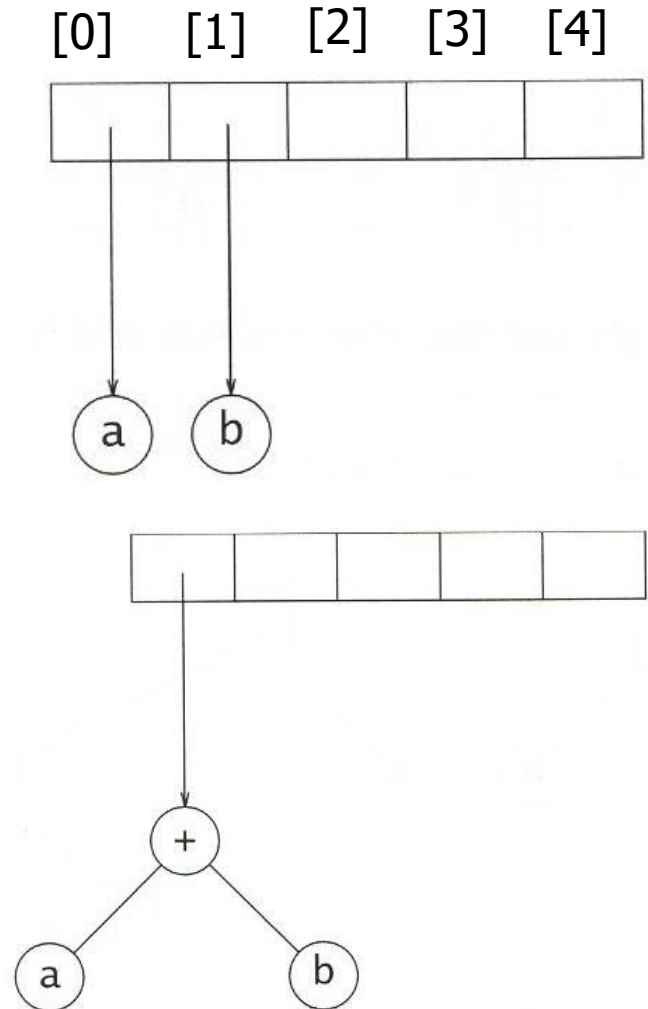
Algorithm to convert a **postfix expression** into an expression tree

- read the expression one symbol at a time.
- if the symbol is operand
 - create a one-node tree
 - push the tree onto a stack
- if the symbol is operator
 - pop two trees T_1 and T_2 from the stack
 - form a new tree whose root is the operator and whose **left** and right subtrees are T_2 and T_1 respectively ตามลำดับ
 - This new tree is pushed onto the stack

Example:

input is ab+cde+**

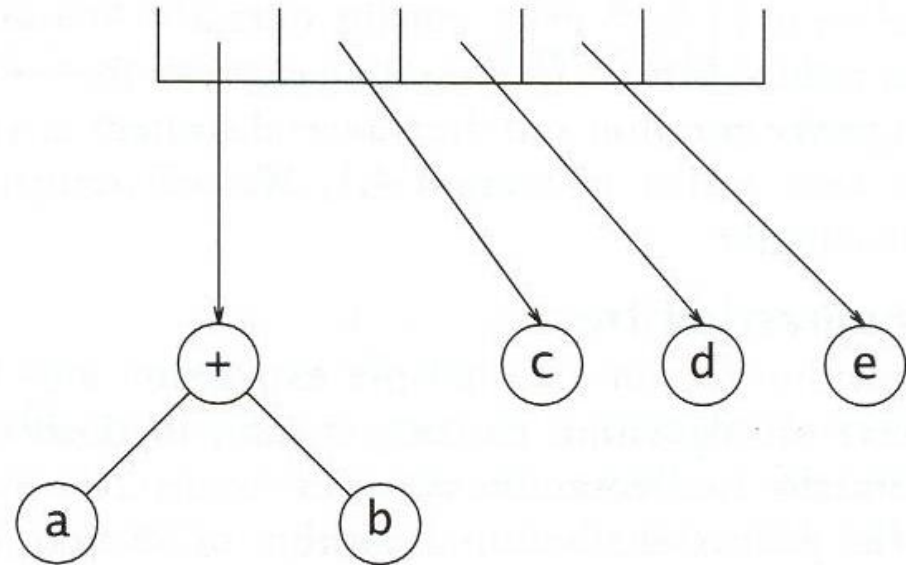
- First two symbols are operands
 - create one-node trees
 - push pointers to them onto a stack
- Next + is read
 - pointers to trees are popped
 - a new tree is formed
 - a pointer to it is pushed onto the stack



Example:

input is $ab+cde+$**

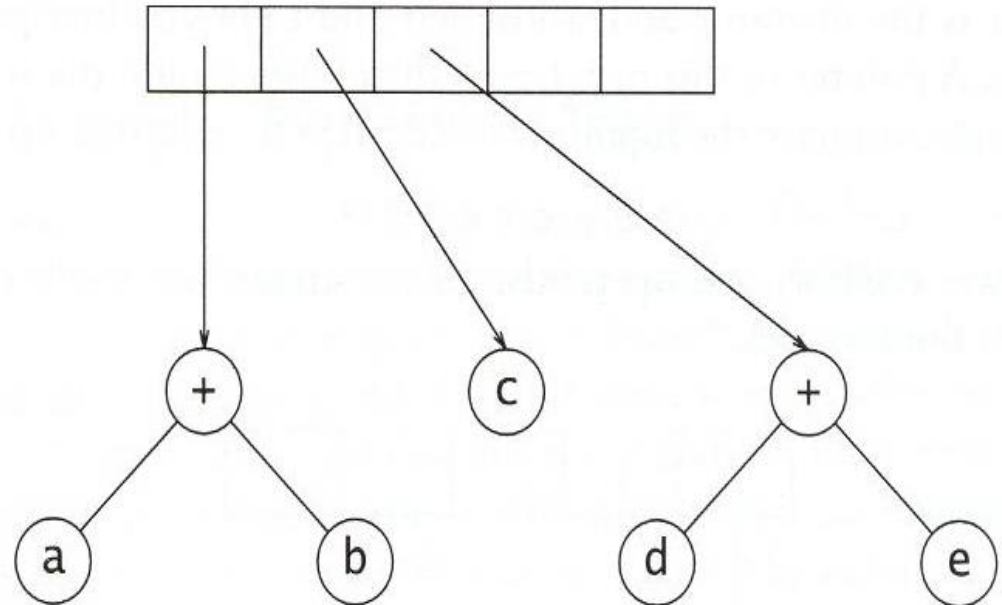
- c, d, e are read
 - for each a one-node tree is created
 - A pointer to the corresponding tree is pushed onto the stack.



Example:

input is $ab+cde+$**

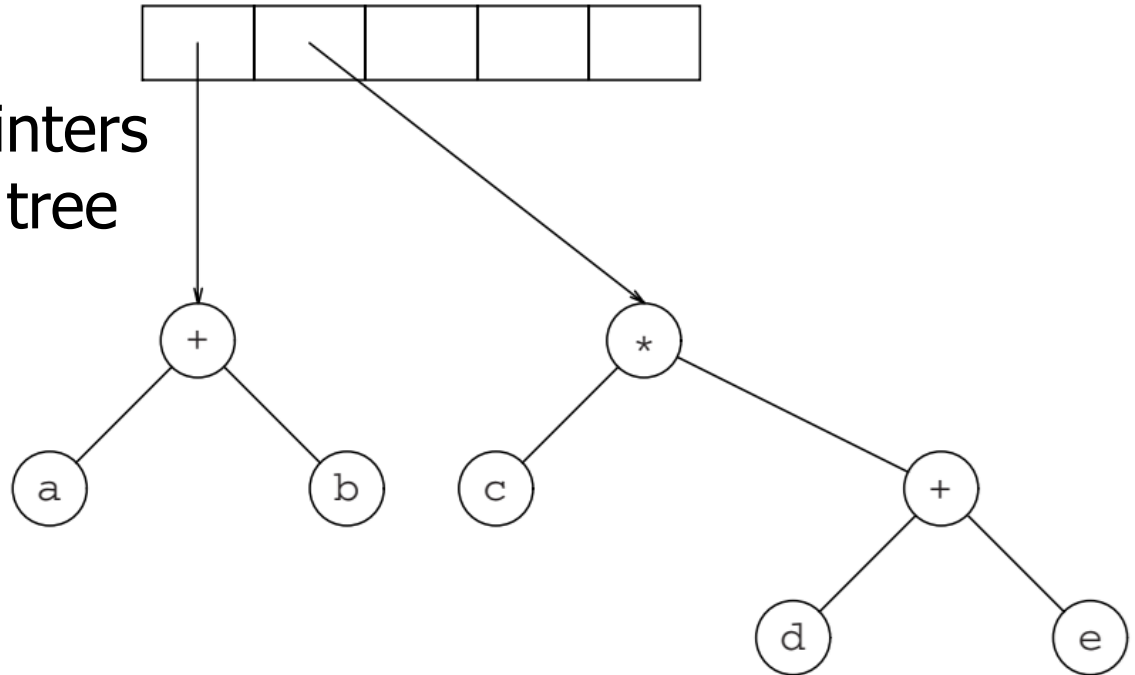
- + is read
 - two trees are merged



Example:

input is $ab+cde+$**

- $*$ is read
 - pop two tree pointers and form a new tree with a $*$ as root



Example:

input is $ab+cde+$**

- Last symbol is read
 - two trees are merged
 - a pointer to the final tree is left on the stack

